



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

Rust 程序设计作业报告

姓名： 杨彩艺

学号： 2412212

2026 年 6 月 3 日

一、功能实现情况

本实验基于 Libra 项目，实现了一个新的命令行功能 `stats`，用于统计当前工作目录中文件类型的分布情况。

1. 基础功能实现

已完成的功能包括：

- (1) 递归遍历当前目录下所有文件
- (2) 按文件扩展名分类统计数量
- (3) 将无扩展名文件归类为 `no_extension`
- (4) 自动忽略 `.libra` 和 `target` 目录
- (4) 支持标准 CLI 调用方式 (`libra stats`)

输出清晰的统计结果

编写了对应的集成测试 (`stats_test.rs`)

2. 工程规范

`cargo` 构建通过，代码格式检查通过，`clippy` 无警告。

```
root@10fd4c93c240:/home# cargo build
Compiling libra v0.17.1208 (/home)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 45.27s
```

```
root@10fd4c93c240:/home# cargo clippy -- -D warnings
Compiling libra v0.17.1208 (/home)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 51.26s
```

二、项目结构

本次实验主要涉及以下文件修改与新增：

libra/

|—— src/

| |—— command/

| |—— stats.rs # 新增: stats 命令实现

| |—— mod.rs # 注册 stats 模块

| |—— cli.rs # 注册 CLI 命令入口

|—— tests/

| |—— command/

| |—— stats_test.rs # 新增: stats 命令测试

| |—— mod.rs # 注册 stats_test

说明:

stats.rs: 核心功能实现

cli.rs: 命令注册入口

stats_test.rs: 功能测试

mod.rs: 模块导入管理

三、设计与实现

1. 整体设计思路

遍历目录 → 提取扩展名 → 统计数量 → 输出结果

核心逻辑: 使用递归方式遍历目录, 然后判断文件类型 (文件 / 目录), 提取扩展名, 如果有扩展名则正常统计, 如果无扩展名则归为 no_extension。最后使

用 BTreeMap 存储统计结果，进而输出统计信息。

2. 关键实现点

- (1) 使用 `std::fs::read_dir` 遍历文件系统
- (2) 使用 `Path::extension()` 提取扩展名
- (3) 使用 `BTreeMap<String, usize>` 统计
- (4) 使用 CLI 输出统一格式

3. 异常情况处理

已考虑以下情况：目录读取失败（I/O 错误处理），文件无扩展名，非 UTF-8 扩展名（安全处理），忽略 `.libra` 和 `target` 目录。

4. 实现截图

运行 `cargo test --test command_test stats` 的结果如下：

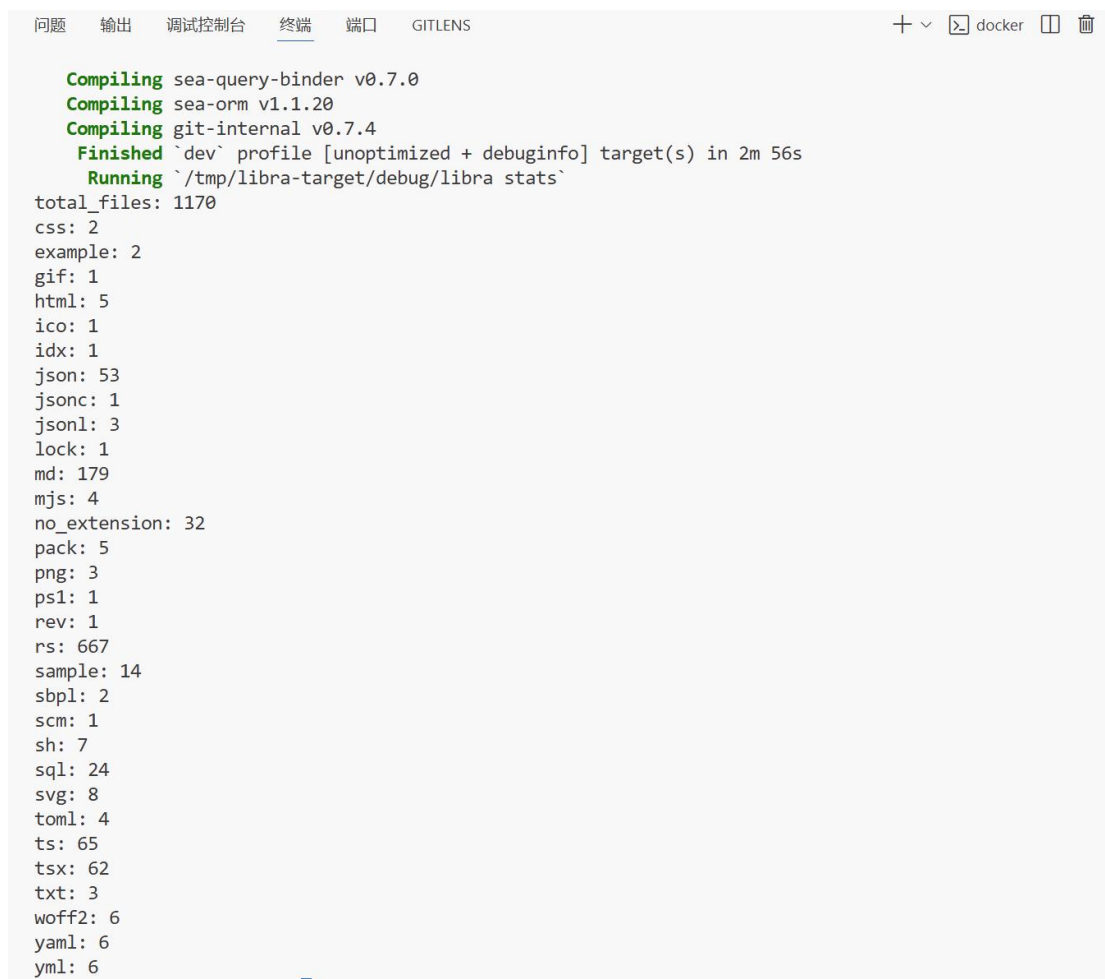
```
Finished `test` profile [unoptimized + debuginfo] target(s) in 13m 19s
Running tests/command_test.rs (target/debug/deps/command_test-1c26ea022930a497)

running 4 tests
test command::stats_test::test_stats_counts_extensions_and_no_extension ... ok
test command::stats_test::test_stats_ignores_libra_and_target_dirs ... ok
test command::diff_test::test_diff_json_output_includes_file_stats ... ok
test command::diff_test::test_diff_stats_count_hunk_lines_that_start_with_header_prefixes ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 1135 filtered out; finished in 12.69s
```

能够正常运行。

运行 `cargo run -- stats`



```
Compiling sea-query-binder v0.7.0
Compiling sea-orm v1.1.20
Compiling git-internal v0.7.4
Finished `dev` profile [unoptimized + debuginfo] target(s) in 2m 56s
Running `/tmp/libra-target/debug/libra stats`
total_files: 1170
css: 2
example: 2
gif: 1
html: 5
ico: 1
idx: 1
json: 53
jsonc: 1
jsonl: 3
lock: 1
md: 179
mjs: 4
no_extension: 32
pack: 5
png: 3
ps1: 1
rev: 1
rs: 667
sample: 14
sbpl: 2
scm: 1
sh: 7
sql: 24
svg: 8
toml: 4
ts: 65
tsx: 62
txt: 3
woff2: 6
yaml: 6
yml: 6
```

如图，可看出 libra stats 命令用于递归统计当前目录中所有文件的扩展名分布情况，并输出各类文件的数量，用于快速分析项目结构，能正常运行。

补充说明：由于 Docker 挂载 Windows 盘 → Rust 增量编译缓存损坏，一开始我直接运行 cargo run -- stats 出现错误，查询资料、询问 AI 后，我执行删除 target : rm -rf target，然后把编译目录移到容器内部

export CARGO_TARGET_DIR=/tmp/libra-target 让 Rust 不再往 /home/target (Windows 盘) 写。

重新编译 + 运行 export LIBRA_SKIP_WEB_BUILD=1, cargo run -- stats。然后得到了预期输出。

四、关键函数说明

在 src/command/stats.rs 中:

1. collect_stats

```
fn collect_stats(root: &Path) -> CliResult<StatsOutput> {
    let mut counts: BTreeMap<String, usize> = BTreeMap::new();
    visit_dir(dir: root, &mut counts)?;

    let total_files: usize = counts.values().sum();

    let extensions: Vec<ExtensionStat> = counts BTreeMap<String, usize>
        .into_iter() IntoIter<String, usize>
        .map(|(extension: String, count: usize)| ExtensionStat { extension, count }) i
        .collect();

    Ok(StatsOutput {
        total_files,
        extensions,
    })
}
```

功能: 遍历目录并统计文件数量, 汇总所有扩展名及对应数量.

2. visit_dir

```
fn visit_dir(dir: &Path, counts: &mut BTreeMap<String, usize>) -> CliResult<()> {
    for entry: Result<DirEntry, Error> in fs::read_dir(path: dir).map_err(stats_read_error)? {
        let entry: DirEntry = entry.map_err(stats_read_error)?;
        let path: PathBuf = entry.path();

        if path.is_dir() {
            if should_ignore_dir(&path) {
                continue;
            }
            visit_dir(dir: &path, counts)?;
        } else if path.is_file() {
            let extension: String = path PathBuf
                .extension() Option<&OsStr>
                .and_then(|ext: &OsStr| ext.to_str()) Option<&str>
                .filter(|ext: &&str| !ext.is_empty()) Option<&str>
                .unwrap_or(default: "no_extension") &str
                .to_string();

            *counts.entry(key: extension).or_insert(default: 0) += 1;
        }
    }

    Ok(())
}
```

功能: 递归遍历目录, 判断文件类型, 更新统计数据.

3. should_ignore_dir

```

fn should_ignore_dir(path: &Path) -> bool {
    matches!(
        path.file_name().and_then(|name| name.to_str()),
        Some(".libra") | Some("target")
    )
}

```

功能：判断是否需要忽略目录，当前忽略：.libra, target

4. render_stats_output

```

2 fn render_stats_output(output: &StatsOutput, writer: &mut impl Write) -> CliResult<> {
3     writeln!(writer, "total_files: {}", output.total_files).map_err(stats_write_error)?;
4
5     for stat: &ExtensionStat in &output.extensions {
6         writeln!(writer, "{}: {}", stat.extension, stat.count).map_err(stats_write_error)?;
7     }
8
9     Ok(())
10 }

```

功能：将统计结果格式化输出，支持标准输出或其他 writer

在 stats_test.rs 中，为验证 stats 命令的正确性，设计了两个集成测试函数，分别用于测试基础功能和特殊情况处理：

1. test_stats_counts_extensions_and_no_extension

```

fn test_stats_counts_extensions_and_no_extension() {
    let temp: TempDir = tempdir().unwrap();

    File::create(path: temp.path().join(path: "a.rs")).unwrap();
    File::create(path: temp.path().join(path: "b.rs")).unwrap();
    File::create(path: temp.path().join(path: "Cargo.toml")).unwrap();
    File::create(path: temp.path().join(path: "README")).unwrap();

    let output: Output = run_libra_command(args: &["stats"], cwd: temp.path());

    assert_eq!(output.status.code(), Some(0));

    let stdout: Cow<'_, str> = String::from_utf8_lossy(&output.stdout);
    assert!(
        stdout.contains("total_files: 4"),
        "unexpected stdout: {stdout}"
    );
    assert!(stdout.contains("rs: 2"), "unexpected stdout: {stdout}");
    assert!(stdout.contains("toml: 1"), "unexpected stdout: {stdout}");
    assert!(
        stdout.contains("no_extension: 1"),
        "unexpected stdout: {stdout}"
    );
}

```

功能说明：

该测试函数用于验证 `stats` 命令对不同扩展名文件的统计能力以及对无扩展名文件的处理情况。具体流程如下：在临时目录中创建多个测试文件，包括：`.rs` 文件（Rust 源文件）、`.toml` 文件（配置文件），无扩展名文件（如 `README`）；

调用 `run_libra_command(&["stats"], temp.path())` 执行命令，获取命令输出结果并进行断言检查，验证内容包括：

总文件数是否正确（`total_files`），各扩展名文件数量是否正确（如 `rs: 2`），无扩展名文件是否被正确归类为 `no_extension`，该测试确保了统计逻辑的基本正确性。

2. `test_stats_ignores_libra_and_target_dirs`

```
fn test_stats_ignores_libra_and_target_dirs() {
    let temp: TempDir = tempdir().unwrap();

    fs::create_dir_all(path: temp.path().join(path: ".libra")).unwrap();
    fs::create_dir_all(path: temp.path().join(path: "target")).unwrap();

    File::create(path: temp.path().join(path: "main.rs")).unwrap();
    File::create(path: temp.path().join(path: ".libra").join(path: "ignored.rs")).unwrap();
    File::create(path: temp.path().join(path: "target").join(path: "ignored.rs")).unwrap();

    let output: Output = run_libra_command(args: &["stats"], cwd: temp.path());

    assert_eq!(output.status.code(), Some(0));

    let stdout: Cow<'_, str> = String::from_utf8_lossy(&output.stdout);
    assert!(
        stdout.contains("total_files: 1"),
        "unexpected stdout: {stdout}"
    );
    assert!(stdout.contains("rs: 1"), "unexpected stdout: {stdout}");
}
```

功能说明：

该测试函数用于验证 `stats` 命令是否能够正确忽略指定目录（如 `.libra` 和 `target`）。具体流程如下：在临时目录中创建 `.libra` 和 `target` 子目录，在这些目录中创建测试文件（应被忽略），在根目录中创建一个正常文件（应被统计），执行 `stats` 命令并获取输出结果。验证内容包括：

总文件数是否只统计非忽略目录中的文件，扩展名统计是否排除了 `.libra` 和 `target` 中的文件，该测试确保了目录过滤逻辑的正确性。

五、扩展功能

本实验只实现了基础功能，并未有更多增加，还未实现支持 `--json`，但在基础功能上可进一步扩展：如支持 JSON 输出（如 `--json`），支持按文件大小统计，支持排序输出（按数量降序）等，当前实现已具备良好的扩展性。

六、总结

通过本次实验，我完成了对 Rust 语言在文件操作、CLI 开发以及工程规范方面的综合实践。在实现过程中，我逐步熟悉了 Rust 标准库中与文件系统相关的操作方法，并掌握了基于 clap 框架开发命令行工具的基本流程。

同时，我对 Rust 项目的模块化设计与代码组织方式有了更清晰的理解，并通过编写和运行 `cargo test` 加深了对集成测试机制的认识。本实验实现的 `stats` 命令能够对项目文件结构进行快速统计与分析，具有一定的实用价值，同时也帮助我更深入地理解了版本管理工具相关功能的实现原理。

此外，本实验在 Docker 容器环境下完成，这一过程对我来说具有一定挑战性。在实践中，我遇到了挂载路径配置错误、容器内代码未更新、以及由于 Windows 文件系统与 Docker 容器之间的兼容性问题导致的 I/O 错误（如 Rust `target` 目录写入失败）等情况。针对这些问题，我通过调整挂载方式、重新构建容器环境，并将编译输出目录重定向至容器内部（如 `/tmp`），成功解决了相关问题。这一过程不仅锻炼了我定位和解决环境问题的能力，也加深了我对容器化开发环境及其文件系统机制的理解。

七、仓库地址

<https://github.com/Y56cy/libra.git>